Technische Universität Berlin
Faculty IV: Electrical Engineering and Computer Science
Freie Universität Berlin
Department of Mathematics and Computer Science
Algorithmic Bioinformatics

welche uni angeben, welche reihenfolge..?

Automatic Optimization of Level Parameters for RSHash

(RSHash is a minimizer based fast, compressed hash table for storing DNA kmers. It was recently introduced by the Reinert Group.)

Bachelor's Thesis

by Bjarne David Jehn

to obtain the degree of "Bachelor of Science" (B. Sc.)
in the Computer Science degree studies (Informatik)

First Reviewer:	Prof. Dr. Knut Reinert
Second Reviewer:	Prof. Dr. Stephan Kreutzer
Supervisor:	Jonas Schulte-Mattler

Declaration of Authorship

I hereby declare that the thesis submitted is my own, unaided work, completed without any external help. Only the sources and resources listed were used. All passages taken from the sources and aids used, either unchanged or paraphrased, have been marked as such.

Where generative AI tools were used, I have indicated the product name, manufacturer, the software version used, as well as the respective purpose (e.g. checking and improving language in the texts, systematic research). I am fully responsible for the selection, adoption, and all results of the AI-generated output I use.

I have taken note of the Principles for Ensuring Good Research Practice at TU Berlin dated 15 February 2023. https://www.static.tu.berlin/fileadmin/www/10002457/K3-AMBl/Amtsblatt_2023/Amtliches_Mitteilungsblatt_Nr._16_vom_30.05.2023.pdf

I further declare that I have not submitted the thesis in the same or similar form to any other examination authority.

Berlin, _____ _____

Date Signature

Abstract

Contents

1 Minimizer Distribution 5

2 MINLP Modeling 6

3 Subsampling 12

1 Minimizer Distribution

The MINLP Modeling only works, if the values $\mathbf{p_K}(m,t)$ are known for every $(m,t) \in \mathcal{M} \times \mathcal{T}$ for the input sequence.

Determining the Distribution

The code to find the minimizer distribution for a given $sequence, k, \mathbf{m}$ and $\mathbf{t}$ follows the following steps:
stats:

1. $count_minimizers$:
   a) set $kmers \leftarrow |sequence| - k + 1$
   b) parse every kmer in the $sequence$ and add all minimizers to a vector $minimizers$
   c) sort $minimizers$
   d) iterate over the sorted vector and
      - count the occurences of each minimizer
      - add all unfrequent minimizers to a new vector $unfrq_minimizers$
      - store their occurences in the vector $counts$ where $counts[i]$ relates to $unfreq_minimizers[i]$
      - store the sum of all occurences in no_skmers
2. $minimizers \leftarrow unfreq_minimizers$
3. $no_minimizers \leftarrow |minimizers|$
4. $r \leftarrow$ elias-fano-compressed bitvector of length $2^{\mathbf{2m}}$ that contains 1s at those positions i, for which the minimizer of value i is present in $minimizers$
5. $s \leftarrow$ corresponding vector S
6. weiter ab Zeile 161

Zeile 48-55: verstehe ich es richtig, dass in der schleife in Zeile 52 jeder minimizer nur einmal zum vektor hinzugefügt wird, auch wenn er mehrere kmere abdeckt?

Distribution on Higher Levels

2 MINLP Modeling

One Level

As a first approach we model the runtime and space consumption for only one level.
Let $\mathcal{M}$ be the Set of all minimizer length and $\mathcal{T}$ the set of all thresholds to be considered.

Runtime

To lookup a k-mer q with minimizer $m(q)$ of frequency $f(m(q))$ on one level, first the bit vector R is checked in order to determin whether $1 \leq f(m) \leq threshold$. This can be ignored when modeling the lookup runtime as this is practically not influenced by the parameters. reason
For $\mathbf{p_K}$ of the k-mers, the corresponding bit in R will be set, which results in a lookup in S. If $m(q)$ does not appear at all or has a $f(m(q)) > t$, a lookup in the hash table will follow. The lookup time T_{lookup} for one level can thus be modeled as:

$$T_{lookup} = T_{SL} * \mathbf{p_K} + T_H * (1 - \mathbf{p_K})$$

In order to determine m and t we introduce binary selector variables μ_m, τ_t for each possible $m \in \mathcal{M}$ and $t \in \mathcal{T}$.
μ_m should be 1 exactly when the optimal minimizer length is m.
τ_t should be 1 exactly when the optimal threshold is t.

Table 2.1: Constants and Variables for One Level

Constants	
T_{SL}	time to lookup a k-mer in vector S
T_H	time to lookup a k-mer in the hash table
$P_K(m,t)$	empirically estimated value for a k-mer with a minimizer of length m to have a frequency less than t
Variables	
$\mathbf{p_K} \in [0,1]$	probability of a k-mer to have a minimizer with frequency $\leq t$
$\mu_m \in \{0,1\}$	binary selector for every minimizer length $m \in \mathcal{M}$
$\tau_t \in \{0,1\}$	binary selector for every threshold $t \in \mathcal{T}$
$\mathbf{m}$	optimal minimizer length
$\mathbf{t}$	optimal threshold

The selectors for m and t must be binary and mutually exclusive:

$$\mu_m \in 0,1 \mid \forall m \in \mathcal{M}$$
$$\tau_t \in 0,1 \mid \forall t \in \mathcal{T}$$
$$\sum_{m \in \mathcal{M}} \mu_m = 1$$
$$\sum_{t \in \mathcal{T}} \tau_t = 1$$

For a fixed minimizer length m and threshold t let $P_K(m,t)$ be the empirically determined percentage of k-mers q that have a minimizer with $f(m(q)) < t$.
We can then model $\mathbf{p_K}$ with:

$$\mathbf{p_K} = \sum_{m \in \mathcal{M}} \left(\mu_m \sum_{t \in \mathcal{T}} P_K(m,t) * \tau_t \right)$$

$\mathbf{m}$ and $\mathbf{t}$ can then be determined by:

$$\mathbf{m} = \sum_{m \in \mathcal{M}} \mu_m * m$$
$$\mathbf{t} = \sum_{t \in \mathcal{T}} \tau_t * t$$

The above results in the following MINLP (Mixed Integer Non Linear Program):

MINIMIZE

$$T_{lookup} = {T_{SL}}^1 * \mathbf{p_K} + T_H * (1 - \mathbf{p_K})$$

SUBJECT TO

$$\mathbf{p_K} = \sum_{m \in \mathcal{M}} \left(\mu_m \sum_{t \in \mathcal{T}} P_K(m,t) * \tau_t \right)$$
$$\mu_m \in 0,1 \mid \forall m \in M$$
$$\tau_t \in 0,1 \mid \forall t \in T$$
$$\sum_{m \in M} \mu_m = 1$$
$$\sum_{t \in T} \tau_t = 1$$
$$\mathbf{m} = \sum_{m \in \mathcal{M}} \mu_m * m$$
$$\mathbf{t} = \sum_{t \in \mathcal{T}} \tau_t * t$$

For now we will assume that $T_{SL} \approx 40ns$ and $T_H \approx 20ns$.
Solving this MINLP will result in a Solution where the μ_m, τ_t corresponding to the smallest $m \in \mathcal{M}$, $t \in \mathcal{T}$ will be set to one.
This is as expected, since this leads to the smallest possible $\mathbf{p_K}$. This minimizes the k-mers found in the vector S and thus maximizes the lookups in the (faster) hash table.

However this also results in a high space consumption, because each entry in the hash table containing the minimizer as a key and the offset as its value is significantly larger than an entry in S which only contains offsets.

In order to take that into account, we have to add a constraint modeling the space consumption.

Space Consumption

The space consumption of the Data Structure used by RSHash can be split into that of the Vectors R, O and S as well as the hash table H.

Bit Vectors R and S

R and S can likely be disregarded when modeling the space consumption.
R is a Bit Vector of length $4^{\mathbf{m}}$ that contains n 1s marking the existence of each encountered minimizer. Using Elias-Fano-Encoding, its size can be compressed to

explain why

possibly model n?

$$n\lceil log(4^{\mathbf{m}}/n)\rceil + 3n$$

Offsetvector O

O contains one entry of size $log|T|$ for the offset of every minimizer with frequency $\leq t$ in T.
Every minimizer of length $\mathbf{m}$ covers an average of $\frac{k-\mathbf{m}+2}{2}$ consecutive k-mers. We can thus model the bits per k-mer in O as:

$$bpk_O = \mathbf{p_K} * \frac{2}{k - \mathbf{m} + 2} * log|T|$$

Hash table H

For $1 - \mathbf{p_K}$ of the k-mers q in T H contains an entry $(key : q, value : offset(q))$
Each entry has size $size(key) + size(value) = 64 + log|T|$.
The bits per k-mer required for the hash table can thus be modeled by

$$bpk_H = (1 - \mathbf{p_K}) * (64 + log|T|)$$

Table 2.2: Constants and Variables for Several Level

Constants	
T_{SL}	time to lookup a k-mer in vector S
T_H	time to lookup a k-mer in the hash table
$P_K(m,t)$	empirically estimated value for a k-mer with a minimizer of length m to have a frequency less than t
Variables	
$\mathbf{p_K^l} \in [0,1]$	probability of a k-mer to have a minimizer with frequency $\leq t$ on level l
$\mu_m^l \in \{0,1\}$	binary selector for every minimizer length $m \in \mathcal{M}$ on level l
$\tau_t^l \in \{0,1\}$	binary selector for every threshold $t \in \mathcal{T}$ on level l
$\mathbf{m}^l$	optimal minimizer length on level l
$\mathbf{t}^l$	optimal threshold on level l

Several Levels

Let

Runtime

RSHash uses several levels, to make use of the skewniss of minimizer distribution.
Each level l has a pair of parameters $\mathbf{m}^l, \mathbf{t}^l$ which result in a probability $\mathbf{p_K^l}$ of a k-mer belonging to level l.
For now, we still assume, that the time for a lookup in the vectors R, S as well as the hash table H are (practically) independent of the parameters $\mathbf{m}^l, \mathbf{t}^l$. reason

When using L levels, the probability of a k-mer belonging to a higher level is dependent on the probability of k-mers belonging to the levels below.
On each level $1 \leq l \leq L$, for $\mathbf{p_K^l}$ of the k-mers, the lookup will be completed by finding a value in the offset vector on that level whilst the remaining $(1 - \mathbf{p_K^l})$ queries result in a lookup on the next level.
We can model the lookup time for all levels $1 \leq l < L$ levels recursively as:

$$T_{lookup}^l = \begin{cases} \mathbf{p_K^l} * T_{SL} + (1 - \mathbf{p_K^l}) * T_{lookup}^{l+1} & | \ 1 \leq l \leq L \\ T_H & | \ l = L + 1 \end{cases}$$

For $L = 3$ levels, this results in:

$$\begin{aligned} T_{lookup} = \quad & \mathbf{p}_\mathbf{K}^1 * T_{SL} \\ & +(1 - \mathbf{p}_\mathbf{K}^1) * \Big(\quad \mathbf{p}_\mathbf{K}^2 * T_{SL} \\ & \qquad +(1 - \mathbf{p}_\mathbf{K}^2) * \Big(\quad +\mathbf{p}_\mathbf{K}^3 * T_{SL} \\ & \qquad\qquad +(1 - \mathbf{p}_\mathbf{K}^3) * T_H \Big) \Big) \end{aligned}$$

Because $\mathbf{m}^l$ and $\mathbf{t}^l$ can be different on every level l, a different set of binary selector variables μ_m^l and τ^l is needed.
The constraints for each level are the same as for one level:

$$\begin{array}{ll} \mu_m^l \in 0, 1 & \mid \forall (l, m) \in (\mathcal{L} \times \mathcal{M}) \\ \tau_t^l \in 0, 1 & \mid \forall (l, t) \in (\mathcal{L} \times \mathbf{t}) \\ \sum_{m \in \mathcal{M}} \mu_m^l = 1 & \mid \forall l \\ \sum_{t \in \mathcal{T}} \tau_t^l = 1 & \mid \forall l \end{array}$$

For every level l $\mathbf{p}_\mathbf{K}^l, \mathbf{m}^l$ and $\mathbf{t}^l$ can also be modeled analogously to having only one level:

$$\begin{aligned} \mathbf{p}_\mathbf{K}^l &= \sum_{m \in \mathcal{M}} \left(\mu_m^l \sum_{t \in \mathcal{T}} P_K(m, t) * \tau_t^l \right) && \mid \forall l \in \mathcal{L} \\ \mathbf{m}^l &= \sum_{m \in \mathcal{M}} \mu_m^l * m && \mid \forall l \in \mathcal{L} \\ \mathbf{t}^l &= \sum_{t \in \mathcal{T}} \tau_t^l * t && \mid \forall l \in \mathcal{L} \end{aligned}$$

this only holds under the assumption, that the minimizer distribution is equal on each level - independent of the choice of m and t. This is not trivial and should be further explored!!

Space Consumption

Offsetvector O

For several levels, the offset vector for level l only contains an entry for a k-mer q, if the k-mers minimizer $m(q)$ has a frequency $f(m(q)) > t^i$ for all levels $1 \leq i < l$ and $f(m(q)) \leq t^l$.
The bits per k-mer for the Offset vector on level l can thus be modeled as:

$$bpk_{O^l} = \prod_{i=1}^{l-1} (1 - \mathbf{p}_\mathbf{K}^i) * \mathbf{p}_\mathbf{K}^l * \frac{2}{k - \mathbf{m}^l + 2} * log|T|$$

Hash table H

The hash table only contains an entry k-mers q, for which $f(m(q)) > t^l$ for all levels $1 \leq l \leq L$.

should I introduce

The bits per k-mer for the hash table can thus be modeled as

$$bpk_H = \prod_{l=1}^{L}(1 - \mathbf{p}_{\mathbf{K}}^{l}) * (64 + log|T|)$$

Limitations

This approach of solving the constructed model has the advantage () of finding optimal results .
However for large genomes, finding the empirical values $P_K(m,t)$ becomes increasingly time consuming and needs a lot of RAM .

paragraph/section about the complexity of finding the solution

For these reasons, an approach yielding a solution that is a certain approximation of the optimum but takes less time to find it could be of interest.
An easy adaption to speed up the search for empirical values would be to only sample part of the genome. By sampling sequences of length at a large number of random locations, the probability of choosing a bad representation becomes highly unlikely.
By sampling only part of the genome, we can speed up the finding of empirical values by a factor of / or even "get it down to a constant" (that is for a fixed number of sample locations - regardless of the genomes size. this however results in a growing error for larger genomes).

over what
does it?
how much, why?
what length
how lorge
factor

3 Subsampling

Motivation

The time for processing a genome in order to find the minimizer distribution $\mathbf{p_K}(m, t)$ for possible minimizer lengths $m \in \mathcal{M}$ and thresholds $t \in \mathcal{T}$ is directly proportional to the size of the genome.
For large genomes, this processing can become the dominating factor when searching for optimal parameters using the MINLP Modeling approach. In order to speed up the search for the empirical values, we introduce a subsampling method.

Subsampling Method

Instead of calculating the distribution over the full genome, we sample n sequences of length $l \geq k$ at random positions in the genome.

how to handle overlapping samples

Time Complexity of Finding the Minimizer Distributions

genaue zeitliche abhängigkeit 1. verstehen und 2. aufschreiben

Time Complexity using Subsampling

By applying the proposed subsampling strategy, the original genome T of length $|T|$ is reduced to the subsampled text T^S of length $|T^S| = n * l$. This means a reduction in size by a factor of $\frac{|T|}{|T^S|}$.
Depending on the choice of n and l this results in a speedup of the processing time by a factor of or even a constant processing time when both parameters are chosen as a constand regardless of $|T|$ (this however should result in a growing error for larger genomes).

assuming no overlaps

factor

time

Error Analysis

Choosing l

The length l has to be at least k.

wenn $l > k$ ist, überlappen sich mehrere kmere an den gesampleten stellen. ich verstehe noch nicht, warum genau dass von vorteil ist.
Weil roling hash - l ca 1000, dann quasi linear

Choosing n

Choosing a constant n would result in a processing time that would no longer be dependent of the genomes length $|T|$, as the sampled text would have constant size $n * l$. For large genomes, the percentage $\frac{|T^S|}{|T|}$ of the genome represented in the subsample can become arbitrarily small, resulting in growing error.
If instead n is chosen relative to $|T|$, $\frac{|T^S|}{|T|}$ will have a constant value independent of the genomes size. Even though this means, that the time needed, in order to find the minimizer distributions would still be proportional to the genomes size, this should be the preferred option as it results in a bounded error of the approximation.
For a large enough n, the probability of choosing a bad representation becomes highly unlikely.

assuming no overlaps

zu vage, begründen